Design Document for Phase 2 of Marketplace Application

Note for the Following Explanations:

At each function, if it is ideal for a return type other than void to be returned but certain conditions may return nothing, then null pointers will be returned as a safe way of returning nothing in C++.

When the word “display” or “input” is used, that refers to text being displayed or inputted from within a terminal interface.

The local storage used by this project to read in example backend data is formatted into text files (accounts.txt, listings.txt, and collections.txt). The program will however write to the transactions.txt file when a session ends.

Class Explanations:

Class Name	Description
FrontEnd	Runs the program’s front end process loop. It will collect inputs from the terminal and present results in an interactive user interface. FrontEnd will call on other classes to perform actions and features present in a marketplace.
Marketplace	This class ensures listings and interactions in the program stay up to date by interacting with memory. Marketplace will be responsible for allowing certain actions, and recording or storing transaction information.
FrontEndFileManager	Responsible for file handling and connecting data sources. Will read in the accounts, ownership recordings, and listings from three different storage text files. Takes care of transaction recording, appending transactions to transactions.txt.
UserSession	Holds current information for what the user is doing on the marketplace program. It will keep tabs on who the user is logged in as, listings available at the time the user logged in, and credit added during the time the user is logged in.

ResultingChange	After an action or operation, gets the result, and stores if the operation was successful, records a description of the event, and records if the session ended.
GameListing	This class holds a video game's information, for games accessible through the marketplace. The information contains the name of the game, the seller and how much it costs.
UserOwnsGame	This class connects a user to the games that they own.
TransactionLog	TransactionLog holds all data that identifies a transaction and its involved parties.
UserAccount	This class holds information independent of each user, and holds data that identifies each user. UserAccount stores the type of user, the credit the user has, and their username.

Functions & Method Explanations:

Method Name	Description	Belongs to Class
executeFrontEnd()	This function loads the accounts, listings, and ownership connections, and runs the terminal command loop up until the user types in "exit" and presses enter. When that happens, transaction records that are pending get written to the log transaction file.	FrontEnd
showStats(action)	Returns a message from an instance of ResultingChange that gets returned by marketplace actions.	FrontEnd
infoMarketplace()	Returns a game title, seller, and price for the listings which are available in the current session the user is in.	FrontEnd
getUserMoney(text)	This turns the front end entered text amount of money that a user enters into an actual	FrontEnd

	number (integer) value that can be accepted into the system. For example, a user entered value like “28.75” would be read in as 2875 integer cents.	
readValueInField(prompt)	Displays a prompt to the user, reads a single line of input from the user, and then returns the user inputted text.	FrontEnd
getFromMarketplace(command)	Decides what to do based on a command entered by user into the terminal, and calls the correct method.	FrontEnd
findListing(title, seller)	Returns the very first listing with a matching title and seller value to the passed through values, returning a GameListing typed object.	Marketplace
findAccount(accountUsername)	Returns the first account matching the passed in accountUsername value, and if none match then a default UserAccount object is returned.	Marketplace
loadData(accounts, listings, collections)	Accounts, game listings, and ownership information is read in from the FrontEndFileManager to ensure most updated data is being shown to the user in the user interface.	Marketplace
logout()	Clears the user’s current session allowing another user to login. Adds “00” to the record, identifying an ended user session.	Marketplace
login(username)	Ensures that the username inputted is tied to an account, and verifies no user session is currently live. If these conditions are met, a session is started for the account.	Marketplace
createAccount(username, type, credit)	Identifies first if the user requesting to create an account is an admin, then checks if the	Marketplace

	requested username being created is valid. If it is, the account is added in the program's memory, with an 01 transaction record being logged.	
deleteAccount(username)	Identifies first if the user requesting to delete an account is an admin, then verifies the username passed through belongs to an account. If yes, the account is deleted and an 02 record is logged, with the admin's session being terminated if it was their own account that was removed.	Marketplace
addToCredit(username, creditAmount)	Verifies the inputted username is tied to an account, and checks that the user is allowed to add credit to the specified account. The amount is checked that it does not interfere with session based limits or balance maximums. The session maximum should not have been reached, and the specified balance limit should not have been reached. If the creditAmount addition falls within the limits, the credit balance is updated and an 06 record is logged.	Marketplace
userBuyGame(gameTitle, seller)	Verifies the user in the current session can buy a game. Checks the user and seller accounts are valid. Verifies the user does not already own the game, and that the buyer and seller have valid credit balances. If conditions are valid, the payment is transferred with ownership added to the user, and an 04 record is logged.	Marketplace
sellGame(gameTitle, price)	Verifies a user can sell games, and that the gameTitle is unique with valid parameters. Including, being less than 25 characters	Marketplace

	long, with a price in the appropriate range, and once the listing is created also logging an 03 record. The method should not make it purchasable in the current session though, the system should wait for users currently logged in to open a new session to see the newly listed game.	
refund(buyer, seller, creditAmount)	Verifies the user actioning the refund is an admin, the the buyer and seller accounts being valid, and that the creditAmount is deductible from the seller, and when added to the buyer will still have a valid balance. The transfer will result in an 05 record being created.	Marketplace
currentAvailableGames()	Returns the listings for games that are buyable and viewable on the marketplace in the current session of the logged in user. Returns nothing if there is no user logged in.	Marketplace
transactions()	Returns transactions that were successful with logged records. The transaction logs should have a correct order of actions logged within them.	Marketplace
resetTransactions()	All transaction log records saved in memory will be removed, so that the next session the user will have fresh memory to store logs reflective of the new session's marketplace.	Marketplace
readListings()	Returns a file of available games, and properly formats the file of records. Information of game names, seller names, and credit prices will have white space removed, and end the file with an "END" identifier.	FrontEndFileManager

readCollections()	Returns a file of ownership entries, containing the owner and the game title from each entry in a collection, removing whitespace in the file, and ending the file with an “END” identifier.	FrontEndFileManager
readAccounts()	Returns a file of game listings, containing records of the game name, seller usernames, and prices from the available games file, removing white space, and ending the file with an “END” identifier.	FrontEndFileManager
logTransactionsFromUser(records)	Records all passed through records to the daily transaction file, keeping the order the records occurred in chronologically, and returns a properly explained fail message if the records are unable to be completely written to the file.	FrontEndFileManager
formatTransaction(record)	One output string is built using the specified transaction’s record code, with values being parsed and displayed correctly, such as displaying a money value as credits and as a readable dollar amount.	FrontEndFileManager
start(account, listings)	When this method runs, the account logged in has its information stored, so that the marketplace shows listings that should be available at the time the user started their session. Credit that could be added is reset to 0 ensuring a fresh session does not yet have a history of credit being added.	UserSession
isUserLoggedIn()	Returns the boolean true if a user is logged in during the current session, and false if the user interacting with the webpage is not logged in.	UserSession

end()	Gets rid of the identity of the currently signed in user, removes starting listings that would show on the homepage, so that the next session can start fresh.	UserSession
creditAddedFor(username)	This method returns the cents and credits that have been added to the currently logged in account, during the current session.	UserSession
wasAvailableWhenLoggedIn(title, seller)	This method checks if the game with the specified title and seller were available when the user started their session.	UserSession
creditAddedTotal(username, creditAmount)	This method keeps track of a total credit value, which will be used as a value that is logged in records like the daily transaction log. It will add creditAmount to a user with a specific username's total credit.	UserSession
formatTransactionRecord()	An output string is built using all transaction data from the current session.	UserSession
canUserBuy()	Returns as true for accounts with type "AA". False otherwise.	UserAccount
canUserSell()	Returns as true for accounts with type "AA", "FS", and "BS". False otherwise.	UserAccount
isAdmin()	Returns as true for accounts with type "AA", "FS", and "SS". False otherwise.	UserAccount

Parameter Variable Types:

The parameter variables for methods do not have defined typed in the UML. Below the typings are declared to provide additional insight into how these parameter variables will be used in the code.

Classes Using Variables	Variables belonging to the Method	Parameter Variables and Types
FrontEnd	getUserMoney, readValueInField, getFromMarketplace	text: string command: string prompt: string
Marketplace	findListing, findAccount	seller: string title: string accountUsername: string
Marketplace	loadData	listings: vector<GameListing> collections: vector<UserOwnsGame> accounts: vector<UserAccount>
Marketplace	login, createAccount, deleteAccount, addToCredit	username: string type: string credit: int creditAmount: int
Marketplace	userBuyGame, sellGame, refund	gameTitle: string seller: string price: int buyer: string credit: int creditAmount: int
UserSession	start, creditAddedFor, wasAvailableWhenLoggedIn, creditAddedTotal	account: UserAccount listings: vector<GameListing> title: string seller: string username: string creditAmount: int
FrontEndFileManager	logTransactionsFromUser, formatTransaction	record: TransactionLog records: vector<TransactionLog>

Input and Outputs Explained:

Below outlines how inputs and outputs for the system will be implemented within the project.

Input / Outputs	Use & Description
std::cin	The main method in the system passes this input to FrontEnd as the type istream&,

	where FrontEnd is able to read in terminal commands.
std::cout	The main method in the system provides this output to FrontEnd as type ostream&, and FrontEnd writes prompts, messages and game listings to it so the terminal can display information to the user.
accounts.txt	The readAccounts method in FrontEndFileManager reads the usernames, accounts and credit value balances from this file.
listings.txt	The readListings method in FrontEndFileManager reads the game names, prices and seller names from this file.
transactions.txt	The logTransactionsFromUser method in FrontEndFileManager will write to and append records to this file.
collections.txt	The readCollections method in FrontEndFileManager reads the owners of a game, and the corresponding game's title.

UML Diagram:

On the next page is the completed UML diagram, showing overall how the system and its components will interact and operate as a marketplace application.

Front End UML for Online Gaming Marketplace

